efrei
INGÉNIEURS - NUMÉRIQUE
PARIS PANTHÉON-ASSAS UNIVERSITÉ

LA GRANDE ÉCOLE D'INGÉNIEURS GÉNÉRALISTES
ET D'EXPERTS DU NUMÉRIQUE
DE L'UNIVERSITÉ PANTHÉON-ASSAS

Programmation Web

CSS

Mohamed HAMIDI
Mohamed.hamidi@efrei.fr
(Basé sur le cours de Maha Naceur, Malika CHARRAD et John Dreyfus)
2025 – 2026

Plan

CSS :

Présentation

Les sélecteurs

Responsive design

CSS
Cascading Style Sheet

- Introduit en 1996
- CSS 1 → CSS 2.0 → CSS 2.1 → CSS 3
- Un langage qui vient compléter HTML
- Gère les mises en forme de votre site
- Permet de choisir les couleurs, sélectionner la police, définir la taille du texte, les bordures, le fond …
- Permet la mise en page de votre site; disposition du menu, sa largeur, la position l'en-tête …

CSS
Cascading Style Sheet

Le code écrit en langage CSS peut être écrit :

- Dans un fichier .css (recommandé)
- Dans l'en-tête <head> du fichier html
- Directement dans les balises du fichiers html via l'attribut style (moins recommandé)

Un fichier texte avec l'extension .css

Se trouve généralement dans le même répertoire que le document qui l'utilise.

OU ajouter du style ?

Styles dans un fichier externe .css en le reliant via la balise link

```
<head>
   <meta charset = "utf-8">
   <title>Linking External Style Sheets</title>
   <link rel = "stylesheet" type = "text/css"
      href = "styles.css">
</head>
```

Avec la balise style donnée dans le head

```
<head>
   <meta charset = "utf-8">
   <title>Embedded Style Sheet</title>

   <!-- this begins the style sheet section -->
   <style type = "text/css">
      em        { font-weight: bold;
                  color: black; }
      h1        { font-family: tahoma, helvetica, sans-serif; }
      p         { font-size: 12pt;
                  font-family: arial, sans-serif; }
      .special { color: purple; }
   </style>
</head>
```

Style intégré directement avec l'attribut style d'une balise

```
<p style = "font-size: 20pt; color: deepskyblue;">
   This text has the <em>font-size</em> and
   <em>color</em> styles applied to it, making it
   20pt and deep sky blue.</p>
```

CSS : exemple

CSS

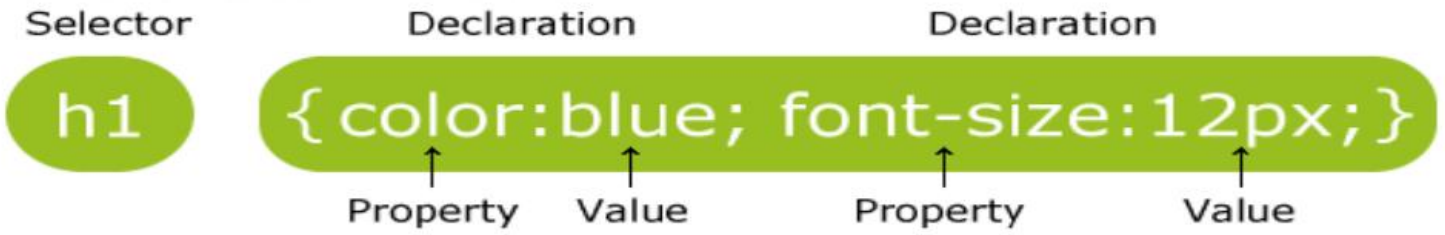
```html
<!DOCTYPE html>
<html>
<head>
<style>
body {
    background-color: #d0e4fe;
}

h1 {
    color: orange;
    text-align: center;
}

p {
    font-family: "Times New Roman";
    font-size: 20px;
}
</style>
</head>
<body>

<h1>My First CSS Example</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

La syntaxe CSS

Exemple

```
p {
  color: red;
  width: 500px;
  border: 1px solid black;
}
```

La syntaxe CSS



- **Sélecteur :** c'est le ou les éléments qui à styler
- **Déclaration :** Une seule règle spécifiant les propriétés de l'élément que vous voulez styliser.
- **Propriétés :** Les moyens par lesquels vous pouvez donner un style à un élément HTML donné.
- **Valeur de la propriété :** choisit l'une des nombreuses apparences possibles pour une propriété donnée,

La syntaxe CSS

- Chaque ensemble de règles doit être entouré d'accolades { }.
- Dans chaque déclaration, utiliser les deux points : pour séparer la propriété de ses valeurs.
- Il faut utiliser un point-virgule ; pour séparer chaque déclaration de la suivante.

Vous pouvez également sélectionner plusieurs éléments et appliquer un seul jeu de règles à chacun d'entre eux. Séparez les sélecteurs multiples par des virgules. Par exemple :

```
p, li, h1 {
  color: red;
}
```

Quelques propriétés CSS

Text Color

```
body {
    color: blue;
}

h1 {
    color: #00ff00;
}

h2 {
    color: rgb(255,0,0);
}
```

Text Alignment

```
h1 {
    text-align: center;
}

p.date {
    text-align: right;
}

p.main {
    text-align: justify;
}
```

Background Color

```
body {
    background-color: lightblue;
}
```

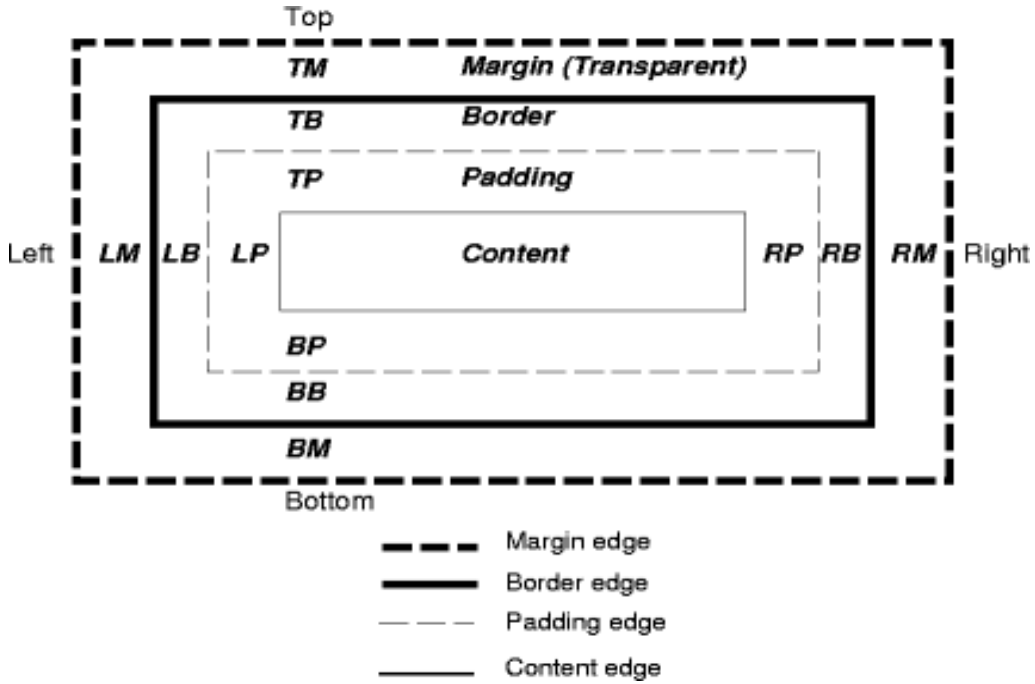
Text Size

```
h1 {
    font-size: 40px;
}

h2 {
    font-size: 30px;
}

p {
    font-size: 14px;
}
```

Text Font

```
p {
    font-family: "Times New Roman", Times, serif;
}
```

Marges, bordures, padding



CSS définit des attributs pour définir précisément ces zones

CSS : les sélecteurs

Un sélecteur de type cible un élément HTML (une balise) de votre document:

- **Les sélecteurs de balises** : il suffit de préciser le nom de la balise html
- **Le sélecteur universel (*)** : permet de sélectionner tout le document
- **Les sélecteurs de classe** : commencent par un point (.)
  - sélectionner tout élément auquel cette classe est appliquée
  - Sélectionner les éléments d'un type donné
  - Sélectionner un élément appartenant à plusieurs classes

CSS : les sélecteurs

- **Les sélecteurs d'ID** : commencent par (#) utilisés de la même manière qu'un sélecteur de classe mais un ID ne peut pas avoir un doublon dans un même document
- **Les sélecteurs d'attributs** : cibler un élément en fonction de la présence d'un attribut unique ou sur des correspondances variées avec la valeur d'un attribut donné

Les sélecteurs

Sélecteurs de balise

HTML

```
<h2> Web Design </h2>
```

CSS

```
<style>
h2 {
  color: white;
}
</style>
```

sélecteur universel

```
* {
  font-family: "Times New Roman";
}
```

Sélecteurs de classe

HTML

class:

```
<h2 class="title"> Web Design </h2>
```

CSS

```
<style>
.title {
  color: white;
}
</style>
```

Sélecteurs d'ID

HTML

id:

```
<h2 id="title"> Web Design </h2>
```

CSS

```
<style>
#title {
  color: white;
}
</style>
```

Sélecteurs avec Combinateur frère general sibling combinator ~

Le combinateur général frère (via le combinateur tilde (~)) sépare deux sélecteurs et **correspond** à toutes les itérations du **deuxième** élément, qui **suivent** le **premier** élément (mais **pas nécessairement immédiatement**), et sont des **enfants** du **même** élément **parent**.

HTML

```html
<article>
  <p>I am the first paragraph.</p>
  <h1>A heading</h1>
  <p>I am a paragraph.</p>
  <div>I am a div</div>
  <p>I am another paragraph.</p>
</article>
```

CSS

```css
p{
  color : red;
}
h1 ~ p {
  font-weight: bold;
  background-color: #333;
  color: #fff;
  padding: .5em;
}
```

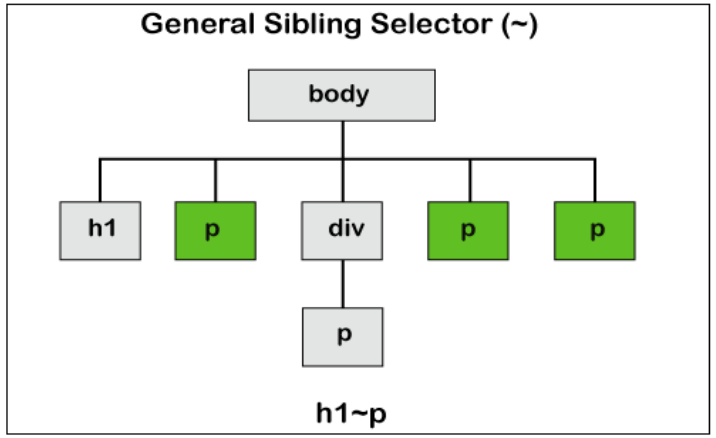
I am the first paragraph.

A heading

I am a paragraph.

I am a div

I am another paragraph.



Sélecteurs avec Combinateur frère adjacent sibling combinator +

Le combinateur frère **adjacent** (+) sépare deux sélecteurs et correspond au **deuxième élément uniquement** qui **suit immédiatement** le premier **élément**, et les deux sont des enfants du même élément parent.

HTML

```html
<h1>A heading</h1>
<p>Hello EveryOne.</p>
<p>Ciao !.</p>
```

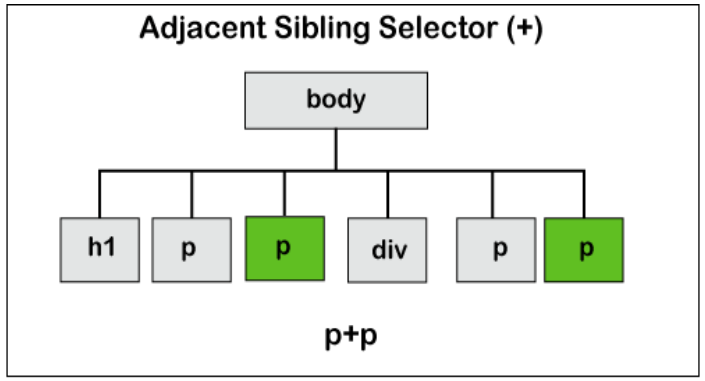
CSS

```css
h1 + p {
  font-weight: bold;
  background-color: #333;
  color: #fff;
  padding: .5em;
}
```

A heading

Hello EveryOne.

Ciao !.



Sélecteurs avec Combinateur enfant child combinator >

Le combinateur enfant (>) est placé entre deux sélecteurs CSS. Il correspond uniquement aux éléments correspondant au second sélecteur qui sont les enfants directs des éléments correspondants au premier. Les éléments descendants plus bas dans la hiérarchie ne correspondent pas.

HTML

```html
<ul>
  <li>Unordered item</li>
  <li>Unordered item
    <ol>
      <li>Item 1</li>
      <li>Item 2</li>
    </ol>
  </li>
</ul>
```

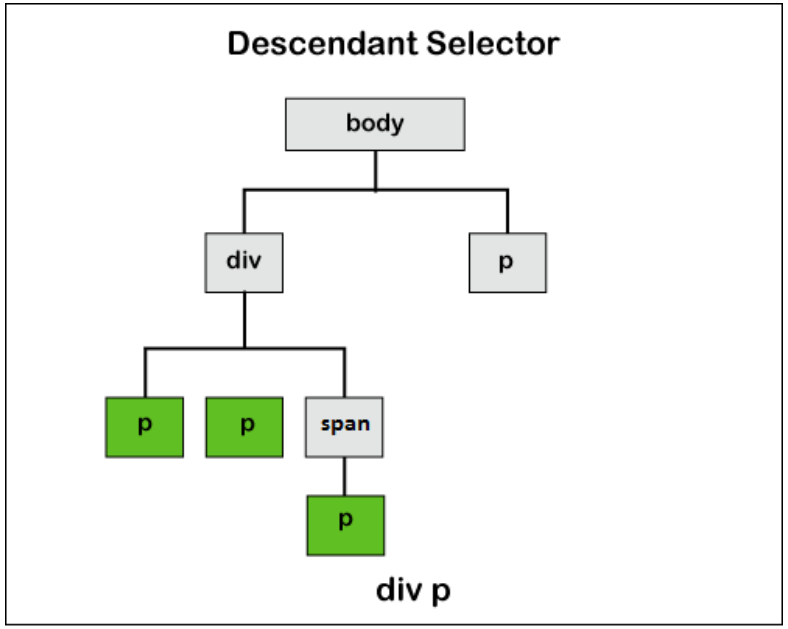
CSS

```css
ul > li {
  border-top: 5px solid red;
}
```

- Unordered item
- Unordered item
  1. Item 1
  2. Item 2

Sélecteur descendant

Utilisé pour faire **correspondre** les éléments **descendants** d'un élément particulier et le représenter à l'aide d'un **espace** unique. Le mot descendant indique que l'élément est imbriqué **n'importe où** dans **l'arborescence** DOM (DOM tree).



HTML

```html
<p>Text free </p>
<div class="box">
  <p>Text in .box</p>
</div>
<p>Text not in .box</p>
```

CSS

```css
.box p {
  color: red;
}
```

Text free

Text in .box

Text not in .box

CSS : Cascade, spécificité et héritage

Objectifs : Contrôler la manière dont le CSS est appliqué au HTML et la manière dont les conflits sont résolus

Pourquoi : Un CSS qui ne s'applique pas comme on le souhaite à un élément !

=> Création de plusieurs règles à appliquer sur un même élément

Solution : Comprendre et appliquer la notion de « Cascade »
=> Quelle règle sera appliquée lorsqu'il y a un tel conflit

La cascade

- Ordre d'apparition des règles dans le CSS

=> 2 règles avec la même spécificité : la dernière sera appliquée

HTML

```html
<body>
  <h1> EFREI - 2027 </h1>
</body>
```

CSS

```css
h1{
  color : yellow;
}
h1{
  color : red;
}
```

color aura comme valeur finale : red

EFREI - 2027

Spécificité

La spécificité mesure combien la sélection est précise :

- **Sélecteur peu spécifique :** cible tous les éléments d'un type donné : score faible
- **Sélecteur plus spécifique :** Ne cible que les éléments dont l'attribut class a la valeur choisie : score important

HTML

```html
<body>
  <h1> EFREI - 2027 </h1>
  <h1 class = 'heading'> Web Programming </h1>
</body>
```

CSS

```css
.heading{
  color : yellow;
}
h1{
  color : red;
}
```

EFREI - 2027

Web Programming

L'héritage

Certaines valeurs pour une propriété CSS se transmettent des éléments parents vers leurs enfants, d'autres non.

HTML

```html
<body>
  <h1> EFREI - <strong>2027</strong> </h1>
</body>
```

CSS

```css
strong{
  color : Blue;
}
h1{
  color : red;
}
```

EFREI - 2027

Attention : Les propriétés telles que largeur, marges, remplissage, et bordures ne se transmettent pas par héritage.

Vue adaptative : la responsivité

Une interface responsive (adaptative) s'adapte à la taille de l'écran, tout en restant confortable à utiliser.

=> Nécessité de concevoir des sites internet capables d'être consultés à partir de n'importe quel terminal, PC de bureau ou téléphone mobile, avec un confort d'utilisation comparable.

La responsivité : les étapes clés

1

Concevoir sa mise en page en prenant en compte les contraintes d'un écran de smartphone

2

Mettre en place des directive viewport dans l'en-tête de toutes les pages.

3

Définir toutes les dimensions horizontales (largeurs, marges droite et gauche...) en unités relatives (ex : pourcentage).

4

Mettre en place des limites minimale et éventuellement maximale.

5

Définir une directive media-query en disposant les éléments au mieux pour un écran plus grand comme celui d'une tablette

6

Définir une autre directive media-query pour disposer les éléments pour un écran de bureau.

Etape 1 : le Viewport

La déclaration **viewport** doit être placée dans TOUTES les pages HTML du site Web, dans la section **head**.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

On définit ici que la largeur de l'écran virtuel doit être identique à celle du device, donc de l'écran physique, et que le zoom doit être de 1 (l'internaute pouvant cependant zoomer ou dézoomer s'il le souhaite).

Etape 2 : définir les dimensions et fixer les limites

1- **Définir les dimensions** en unités relatives : Pour les marges, dimensions des éléments, etc. L'unité la plus couramment utilisée est le pourcentage

-> Ainsi toutes les dimensions sont proportionnelles à celles de l'écran,

2- **Fixer des limites** : 4 propriétés permettent de fixer les limites min-width, max-width, min-height et max-height

-> Contrairement aux autres dimensions, ces valeurs seront définies en unités absolues (en pixels par exemple)

Les média-queries

Les media-queries permettent, tout en restant sur un terminal du type écran, de défiltrer les styles en fonction de la taille de cet écran.

@media associe des règles CSS à un périphérique particulier. En utilisant cette directive, il est donc possible de définir des mises en page spécifiques à chaque périphérique : écran, imprimante, smartphone, etc.

```
/*Rules to apply if a screen is visited*/
@media screen {
    h1{
        font-size: 24pt;
        font-weight: normal;
    }
    nav{
    display : inline-block;
    }
}

/*Rules to apply if the page will be printed*/
@media print {
    h1{
        font-size: 18pt;
        font-weight: bold;
    }
   nav{
    display : none;
   }
}
```

```
<head>
    <meta charset='utf-8'>
    <title>Page Title</title>
    <meta name='viewport' content='width=device-width, initial-scale=1'>
    <link rel='stylesheet' type='text/css' media='screen' href='style.css'>
    <link rel='stylesheet' type='text/css' media='print' href='print.css'>
</head>
```

Les flexbox

Les flex-box ou les boites flexibles, est une nouvelle propriété introduite en CSS3.

Elles rendent la conception adaptative bien plus facile.

```
.box {
        display : flex;
        flex-direction: row-reverse;
}
```

```
<div class="box">
        <div>One</div>
        <div>Two</div>
        <div>Three</div>
</div>
```

Les flexbox

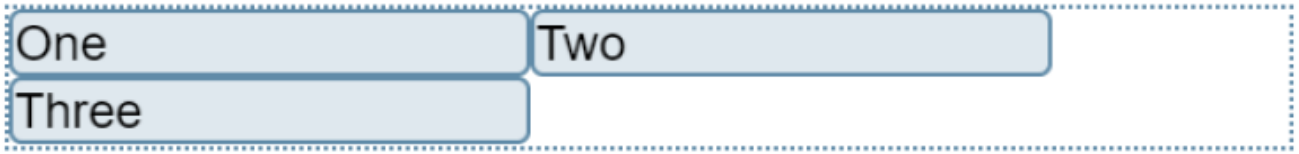
Avec **flex-wrap**, si les éléments sont trop grands pour tenir sur une seule ligne, ils passeront sur une autre ligne

```
.box {
        display : flex;
        flex-wrap: wrap;
}
```

css

```
<div class="box">
        <div>One</div>
        <div>Two</div>
        <div>Three</div>
</div>
```

html

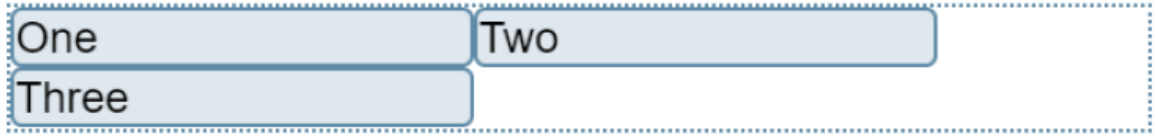


Les flexbox

Flex-flow : propriété raccourcie de flex-direction and flex-wrap

```
.box {
        display : flex;
        flex-flow: wrap;
}
```

```
<div class="box">
        <div>One</div>
        <div>Two</div>
        <div>Three</div>
</div>
```



Lectures obligatoires

- MDN : les bases du CSS
- Utiliser CSS dans une page WEB
- Les sélecteurs de classe
- Les sélecteurs d'ID
- Les pseudo-classes

Codepen : Exemple complet du cours

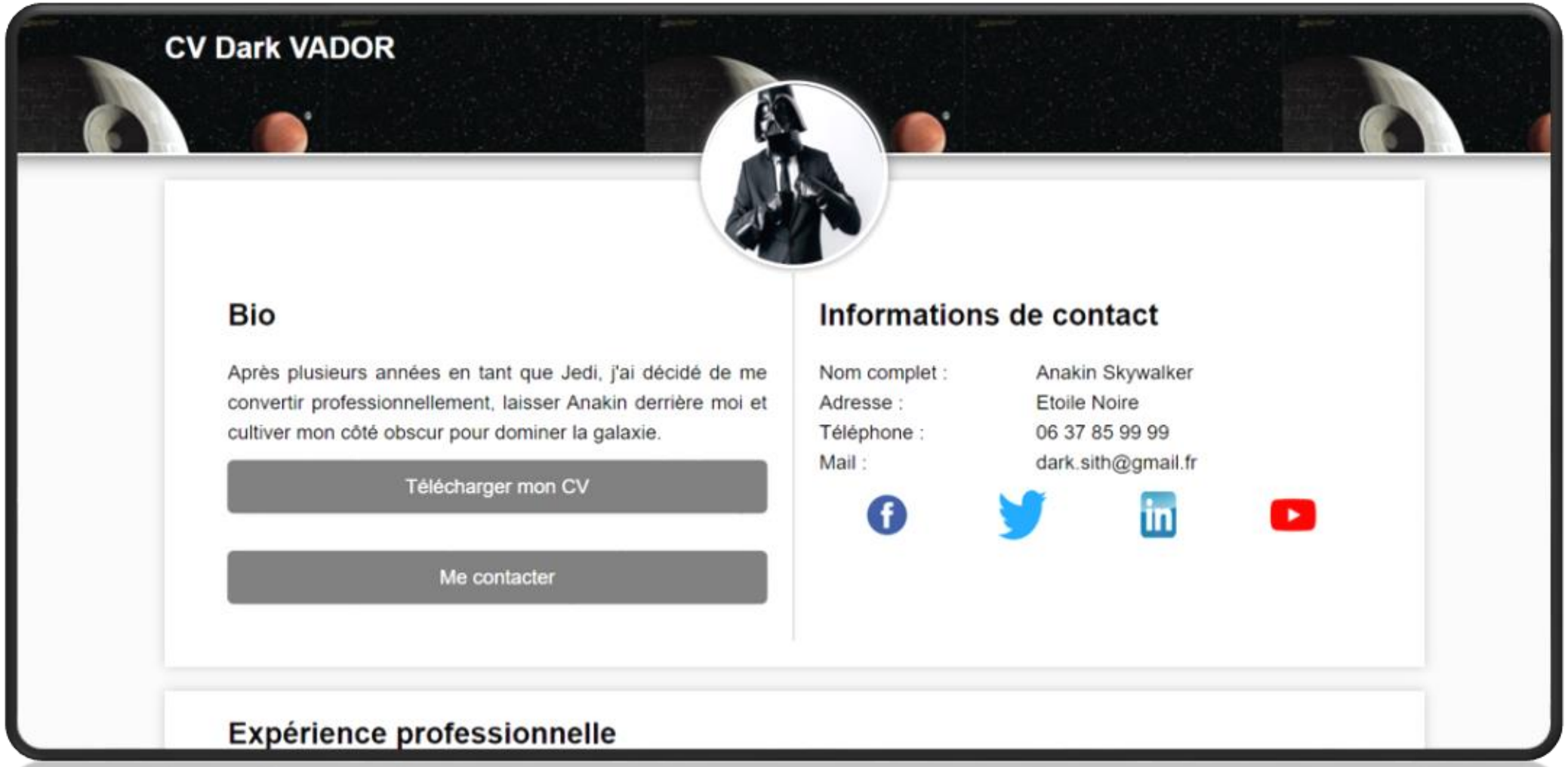
Lien Codepen : https://codepen.io/mohamed-hamidi/pen/WNqzMpZ

Atelier – Partie 2

On reprend le CV que vous avez réalisé à la partie 1. Vous allez maintenant utiliser les différentes notions étudiées afin de styliser votre CV en ligne.

Objectif : ajouter du style à la page du CV créée et la rendre adaptative

Travail à faire :

- Mettre le titre du CV en forme (couleur, police, taille, etc)
- Styliser votre photo (bordure, taille, etc)
- Ajouter une couleur différente à chaque section
- Amusez-vous avec les propriétés étudiées : pensez à utiliser les sélecteurs de classe et d'attributs !

Dark Vador a aussi joué le jeu !